DESIGN AND FPGA IMPLEMENTATION OF AN FFT-BASED INDUSTRIAL SIGNAL PROCESSING SYSTEM FOR MACHINE FAULT DETECTION USING MACHINE LEARNING

PROJECT REPORT

Submitted by

HARINI M (241EC165)

NAVEENRAJ S (241EC233)

MUTHU KUMAR J (241EC228)

In partial fulfilment for the award of the degree

BACHELOR OF ENGINEERING

In

ELECTRONICS AND COMMUNICATION ENGINEERING



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)

SATHYAMANGALAM-638401

ANNA UNIVERSITY: CHENNAI 600025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**Design and FPGA Implementation of an FFT-Based Industrial Signal Processing System for Machine Fault Detection Using Machine Learning**" is the Bonafide work of "HARINI M(241EC165) NAVEENRAJ S (241EC233) and MUTHU KUMAR J (241EC228)"who carried out the project work under my supervision.

Dr. Prakash S P

HEAD OF THE DEPARTMENT

Department of Electronics and Communication Engineering

Bannari Amman Institute of Technology

Dr. Sajan P Philip

ASSOCIATE PROFESSOR

Department of Electronics and Communication Engineering

Bannari Amman Institute of Technology

Submitted for Project Viva Voice examination held on……

Internal Examiner **External Examiner**

DECLARATION

We affirm that the project work titled **"Design and FPGA Implementation of an FFT-Based Industrial Signal Processing System for Machine Fault Detection Using Machine Learning"** being submitted in partial fulfillment for the award of the degree of **Bachelor of Engineering** in **Electronics and communication engineering** is the record of original work done by us under the guidance of **Dr. Sajan P Philip**, Associate Professor**,** Department of Electronics and Communication Engineering. It has not formed a part of any other project work(s) submitted for the award of any degree or diploma,either in this or any other University.

HARINI M	**NAVEENRAJ S**	**MUTHU KUMAR J**
(7376231EC154)	**(7376231EC237)**	**(7376231EC228)**

I certify that the declaration made above by the candidates is true.

Dr. Sajan P Philip

ACKNOWLEDGEMENT

We would like to enunciate heartfelt thanks to our esteemed Chairman **Dr. S.V. Balasubramaniam**, and the respected Principal **Dr. C. Palanisamy,** for providing excellent facilities and support during the course of study in this institute.

We are grateful to **Dr. Prakash S P**, **Head of the Department, Department of Electronics and Communication engineering,** for his valuable suggestions to carry out the project work successfully.

We wish to express our sincere thanks to Faculty guide **Dr. Sajan P Philip , Associate Professor**, **Electronics and communication engineering,** for his constructive ideas, inspirations, encouragement, excellent guidance, and much needed technical support extended to complete our project work.

We would like to thank our friends, faculty and non-teaching staff who have directly and indirectly contributed to the success of this project.

HARINI M (241EC165)

NAVEENRAJ S (241EC233)

MUTHU KUMAR J (241EC228)

ABSTRACT

Industrial machine condition monitoring plays an important role in identifying abnormal operating conditions and reducing unexpected equipment failures. Digital signal processing and machine-learning techniques can be used to analyze machine signals and identify potential faults. This project presents the design and implementation of an FPGA-based signal-processing system using an Analog-to-Digital Converter (ADC), a moving average filter, and a Fast Fourier Transform (FFT). The analog input signal is acquired by the AD7991 ADC through the Inter-Integrated Circuit (I^2C) protocol and converted into digital samples. These samples are processed using a 12-point moving average filter to reduce short-term variations, followed by a 16-point Radix-2 Decimation-in-Time (DIT) FFT for frequency-domain analysis. The processed digital data is then transmitted to the DAC121S101 Digital-to-Analog Converter through the Serial Peripheral Interface (SPI) protocol and converted into an analog output signal, which is observed using an oscilloscope. In addition, a separate machine-learning workflow is developed using a dataset and Python for data preprocessing, model training, testing, and prediction of machine operating conditions or fault categories. The FPGA-based signal-processing system and the machine-learning model are implemented as separate paths in the current project stage. Verilog HDL and Xilinx Vivado are used for FPGA design and verification, while Python-based tools support machine-learning operations. The project demonstrates the application of FPGA-based signal processing and dataset-based machine learning for industrial condition monitoring and machine fault detection.

Keywords: FPGA, I^2C, SPI, ADC, DAC, moving average filter, FFT, machine fault detection, machine learning, Verilog H

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	BONAFIDE CERTIFICATE	**ii**
	DECLARATION	**iii**
	ACKNOWLEDGEMENT	**iv**
	ABSTRACT	**v**
	TABLE OF CONTENTS	**vi**
	LIST OF FIGURES	**x**
	LIST OF TABLES	**ix**
	LIST OF ABBREVIATIONS	**xi**
1	**INTRODUCTION**	**1**
1.1	BACKGROUND OF THE PROJECT	1
1.2	OVERVIEW OF THE PROPOSED SYSTEM	3
2	**LITERATURE SURVEY**	**7**
2.1	FPGA-BASED DIGITAL FILTERING	7
2.2	FPGA-BASED FFT PROCESSOR	8
2.3	FPGA-BASED FREQUENCY ANALYSIS	8
2.4	OPTIMIZATION OF FFT HARDWARE ARCHITECTURE	9
2.5	SUMMARY OF THE LITERATURE	9
3	**OBJECTIVES AND METHODOLOGY**	**11**
3.1	OBJECTIVES OF THE PROPOSED WORK	11

3.2	SYNTHETIC PROCEDURE AND FLOW DIAGRAM OF THE PROPOSED WORK	12
3.2.1	FLOWDIAGRAM	13
3.2.2	EXPLANATION OF EACH BLOCK	14
3.3	SELECTION OF TOOLS, COMPONENTS, TECHNIQUES, AND PROCEDURES	16
3.3.1	AD7991 ANALOG-TO-DIGITAL CONVERTER	16
3.3.2	I²C COMMUNICATION PROTOCOL	17
3.3.3	12-POINT MOVING AVERAGE FILTER	17
3.3.4	16-POINT RADIX-2 DIT FFT ALGORITHM	17
3.3.5	DAC121S101 AND SPI COMMUNICATION	18
3.3.6	FPGA DEVELOPMENT BOARD AND OSCILLOSCOPE	18
3.3.7	MACHINE-LEARNING TECHNIQUE FOR FAULT DETECTION	18
4	**PROPOSED WORK**	**19**
4.1.1	OVERALL DESIGN ARCHITECTURE	19
4.1.2	SIGNAL ACQUISITION USING THE AD7991 ADC	19
4.1.3	INTEGRATION OF THE MOVING AVERAGE FILTER	20
4.1.4	INTEGRATION OF THE 16-POINT RADIX-2 DIT FFT	20
4.1.5	FFT FEATURE EXTRACTION AND DATASET GENERATION	21
4.1.6	MACHINE-LEARNING DATASET PREPROCESSING	21
4.1.7	FAULT CLASSIFICATION	21
4.2	METHODOLOGY OF THE PROPOSED WORK	22
4.2.1	SYSTEM DESIGN AND DATA-FLOW DEFINITION	22

4.2.2	RTL DEVELOPMENT AND MODULE INTEGRATION	23
4.2.3	SIMULATION AND FUNCTIONAL VERIFICATION	23
4.2.4	FPGA SYNTHESIS AND IMPLEMENTATION	24
4.2.5	HARDWARE TESTING AND OSCILLOSCOPE OBSERVATION	24
4.2.6	MACHINE-LEARNING TRAINING AND EVALUATION	25
4.2.7	RESULT ANALYSIS AND DOCUMENTATION	25
5	**RESULTS AND DISCUSSION**	**26**
5.1	SIMULATION RESULTS	26
5.2	MOVING AVERAGE FILTER RESULTS	26
5.3	FFT PROCESSING RESULTS	27
5.4	FPGA IMPLEMENTATION RESULTS	27
5.5	MACHINE-LEARNING RESULTS	29
5.6	DISCUSSION OF RESULTS	29
6	**CONCLUSION AND FUTURE SCOPE**	**30**
6.1	CONCLUSION	30
6.2	LIMITATIONS	30
6.3	FUTURE SCOPE	31
6.4	FINAL SUMMARY	31
	REFERENCES	**32**
	APPENDICES	**33**
	INDIVIDUAL CONTRIBUTION	**34**
	ORIGINALITY SCORE	**35**

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
3.1	HARDWARE COMPONENTS USED	16

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1.1	BLOCK DIAGRAM OF DSP SYSTEM	3
1.2	ON-FPGA FFT BLOCK DIAGRAM	4
3.1	FLOW DIAGRAM OF THE PROPOSED WORK	13
4.1	OVERALL DESIGN ARCHITECTURE OF THE PROPOSED SYSTEM	20
5.1	VIVADO SIMULATION RESULT	27
5.2	MOVING AVERAGE FILTER RESULT	27
5.3	FFT PROCESSING RESULT	28

LIST OF ABBREVIATIONS

ADC	Analog to Digital Converter
DAC	Digital to Analog Converter
DIT	Decimation in Time
DFT	Discrete Fourier Transform
DSP	Digital Signal Processing
FFT	Fast Fourier Transform
FPGA	Field Programmable Gate Array
HDL	Hardware Description Language
I^2C	Inter Integrated Circuit
ML	Machine Learning
PMOD	Peripheral Module
RTL	Register Transfer Level
SDA	Serial Data
SCL	Serial Clock
SPI	Serial Peripheral Interface
SNR	Signal to Noise Ratio
USB	Universal Serial Bus
VHDL	VHSIC Hardware Description Language
VHSIC	Very High Speed Integrated Circuit

CHAPTER – I

INTRODUCTION

Digital signal processing (DSP) is fundamental to modern electrical and computer engineering, facilitating the mathematical manipulation, filtering, and transformation of physical signals converted into discrete-time digital representations. Processing signals digitally enables precise feature extraction, noise suppression, and reliable system state synthesis immune to thermal drift. These capabilities make dsp indispensable for wireless communications, audio equipment, medical imaging, and real-time industrial condition monitoring. In predictive maintenance frameworks, real-time signal extraction is critical for evaluating mechanical health and preventing catastrophic equipment failure. This project presents the architecture, rtl design, simulation, and fpga implementation of a complete digital signal processing chain engineered to condition raw analog inputs, eliminate high-frequency interference, and identify dominant spectral components in real time. The physical acquisition interface utilizes a high-speed analog-to-digital converter (ADC) that samples continuous time-domain input voltages into binary data. To address dc offsets caused by sensor biasing or baseline shifts, the pipeline routes raw samples through a dedicated dc offset removal block, preserving dynamic range and preventing spectral artifacts. Following dc calibration, a hardware-efficient moving average filter smooths high-frequency white noise and sensor jitter. Filtered data streams are buffered in dedicated memory blocks to form contiguous frames for batch spectral analysis using a 1024-point fast fourier transform (FFT) engine. The FFT engine reduces computational complexity from o(n^2) to o(n log n) by executing butterfly computations with complex twiddle factors, transforming time-domain frames into complex frequency-domain representations. a magnitude calculation module evaluates euclidean magnitude or its approximation across discrete frequency bins. downstream peak-detection logic isolates the dominant frequency component to identify operational

fundamental frequencies or fault signatures. Additionally, a digital-to-analog converter (DAC) controller converts processed digital waveforms back into continuous physical voltages for closed-loop monitoring. To ensure deterministic execution with sub-microsecond latency, the architecture is implemented on field-programmable gate array (FPGA) hardware using verilog HDL. fpgas leverage customized hardware parallelism for pipelined fft butterfly execution and deterministic data flow. comprehensive rtl simulations verify numerical precision, evaluate signal-to-noise performance, and validate dominant frequency detection prior to hardware deployment.

1.1 BACKGROUND OF THE PROJECT

Digital signal processing (DSP) forms the cornerstone of modern industrial condition monitoring and automated diagnostic frameworks. physical signals harvested from industrial machinery such as mechanical vibrations, acoustic emissions, or motor currents—are inherently continuous in time and highly susceptible to ambient environmental noise, sensor degradation, and static dc offsets. to preserve diagnostic integrity, signal conditioning and digital filtering are critical preliminary stages prior to frequency-domain analysis. In the proposed pipeline, a hardware-efficient moving average filter mitigates high-frequency sensor jitter and transient noise by computing the unweighted average of consecutive discrete samples. Once smoothed, continuous streams of discrete data points are processed via the fast fourier transform (FFT) algorithm, which drastically lowers computational complexity to transform time-series waveform data into its constituent complex spectral components.Field-programmable gate arrays (FPGA) provide a dedicated hardware fabric capable of fine-grained spatial parallelism and pipelined dataflows. By synthesizing custom register-transfer level (RTL) architectures directly onto the fpga, the system executes real-time filtering, spectral decomposition, and fault-signature extraction with deterministic, low-latency performance essential for early machine fault detection

1.1.1 NEED FOR DIGITAL SIGNAL PROCESSING

The primary objective of this project is to model, design, and validate an integrated hardware-

oriented digital signal processing chain for real-time industrial condition monitoring. In automated machine fault detection, raw sensor signals captured from rotating machinery—such as accelerometers and vibration transducers—are frequently corrupted by high-frequency white noise, transient interference, and persistent dc baseline offsets. To establish high-fidelity spectral analysis, the system first executes dc offset cancellation and applies a hardware-efficient moving average filter to attenuate unwanted noise variations while preserving critical transient dynamics. Subsequently, the smoothed time-domain sequence is transformed into the frequency domain using a 1024-point fast fourier transform (FFT) algorithm. By performing radix-based butterfly operations, the fft engine efficiently extracts the complex spectral distribution of the signal, enabling a dedicated magnitude calculation unit and peak-detection module to isolate dominant fault frequencies and harmonic signatures essential for early predictive maintenance and automated diagnosis. Figure 1.1 illustrates the fundamental dsp pipeline concept.

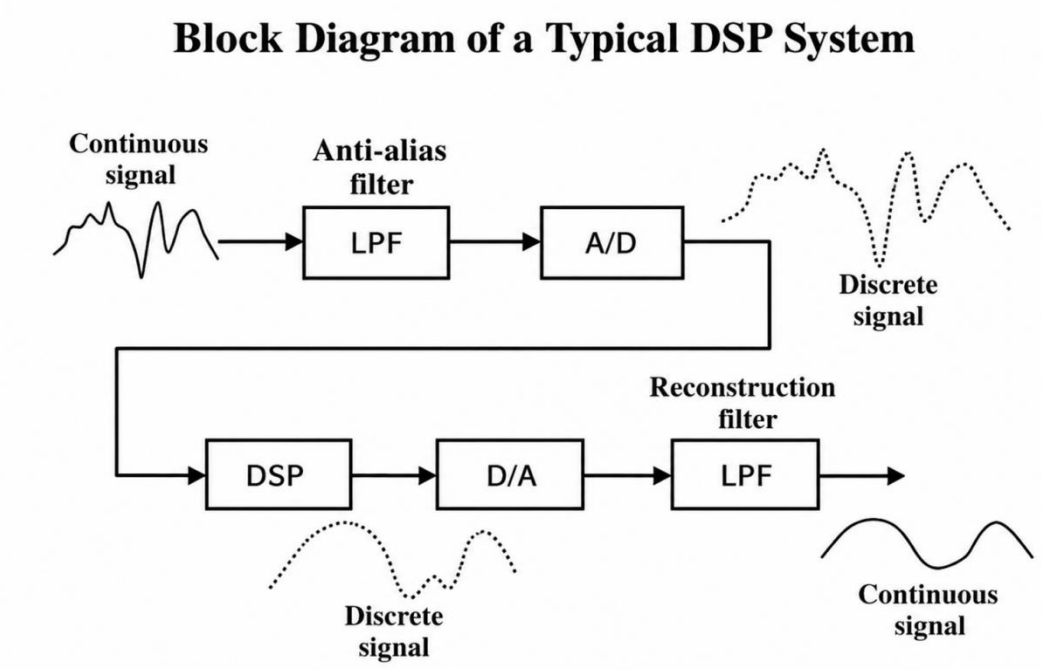


FIGURE 1.1 BLOCK DIAGRAM OF DSP SYSTEM

1.2 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system consists of several modules connected in sequence. The input signal is converted into digital samples using simulated adc data. The input-conditioning module removes the dc offset by subtracting 2048 from the adc values. The conditioned samples are passed through a moving average filter to reduce unwanted variations. The filtered samples are stored in an fft buffer, which collects 1024 samples for processing. The fft processor converts the samples from the time domain to the frequency domain using butterfly operations and twiddle factors. The fft magnitude module calculates the strength of each frequency component using the real and imaginary outputs. The dominant-frequency detection module identifies the bin with the highest magnitude and calculates its corresponding frequency. Finally, the dac interface represents the conversion of a digital output into an analog voltage. The complete proposed system is shown in figure 1.2.

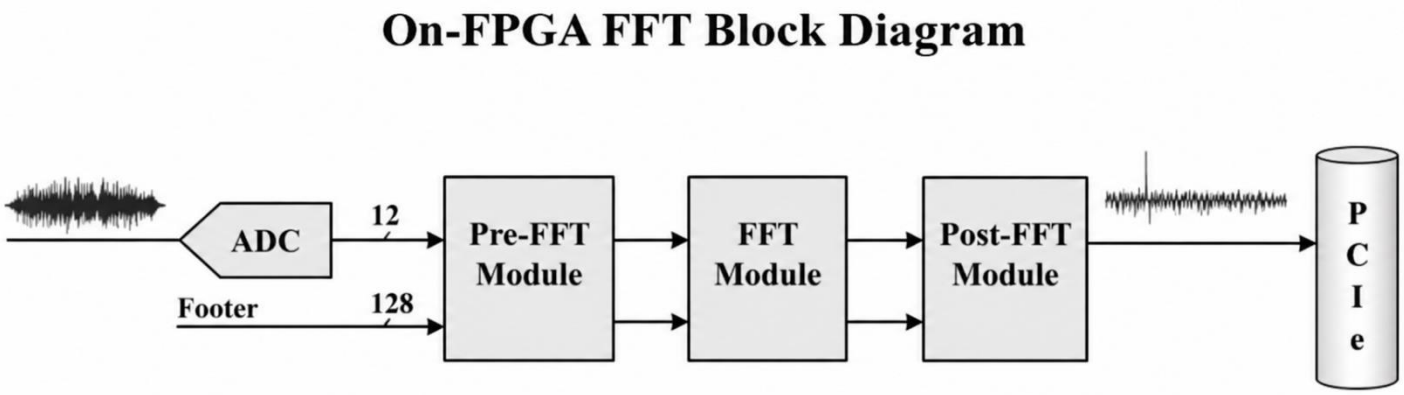


FIGURE 1.2 On-FPGA FFT Block Diagram

1.2.1 SIMULATION-BASED ANALYSIS METHODOLOGY

To rigorously validate the functional correctness and numerical precision of the custom rtl pipeline prior to target physical synthesis and fpga configuration, a comprehensive end-to-end simulation testbench was constructed. System-level verification is conducted at the register-transfer level (RTL) using modelsim/vivado simulator, evaluating fixed-point digital representations at every critical intermediate interface. The testbench models a synthetic continuous-time input signal containing fundamental fault harmonics, sensor noise, and dc bias, passed through an integrated behavioral analog-to-digital converter (ADC) model.

Digital waveform sequences are continuously monitored across key architectural observation points: raw 12-bit adc integer codes, baseline-calibrated signed samples following dc offset subtraction, smoothed output streams generated by the moving average filter module, and the 1024-point complex spectral outputs emitted by the fft engine. spectral bin frequencies are deterministically mapped using the relationship $\delta f = fs / n$, where fs represents the system sampling frequency (10 khz) and n denotes the fft frame size (1024 points). This configuration yields a precise discrete frequency resolution of approximately 9.765 hz per bin across the 0 to 5 kHz nyquist bandwidth.Simulation outputs are captured, tabulated, and analyzed to evaluate the dynamic signal-to-noise ratio (SNR), word-length truncation noise, and harmonic distortion suppression characteristics across varying input profiles. Tabulated numerical metrics serve as a formal baseline for verifying algorithm fidelity in simulation environments, establishing clear verification boundaries between software-simulated functional validation and eventual hardware-in-the-loop physical fpga board deployment.

1.2.2 SCOPE AND OBJECTIVES OF THE PROJECT

The primary scope of this project encompasses the architectural specification, verilog rtl implementation, functional verification, and resource optimization of a modular digital signal processing system dedicated to industrial predictive maintenance. the system design integrates front-end signal conditioning (dc offset correction), low-pass spatial noise smoothing (moving average filtering), frame-based memory buffering, 1024-point radix-2/radix-4 fft spectral transformation, complex magnitude evaluation, and high-speed automated peak-frequency detection logic within a unified hardware pipeline. Through this execution, the project provides in-depth technical insights into fixed-point arithmetic optimization, pipeline synchronization, memory bandwidth management using block rams, and low-latency peak extraction for real-time fault identification. Future expansions of this work involve physical hardware target implementation on modern fpga development boards (e.g., xilinx artix-7/zynq), interfacing hardware adcs/dacs via spi/i2s interfaces, incorporating lightweight machine learning classifiers (such as svm or shallow neural networks) directly into the hardware fabric, and integrating high-speed pcie/uart visual spectrum telemetry for real-time industrial health monitoring.

CHAPTER II

LITERATURE SURVEY

The literature survey systematically reviews key foundational research across real-time digital signal processing, hardware-optimized digital filtering, fast fourier transform (FFT) algorithms, and reconfigurable fpga implementations for industrial machine fault detection. early condition-monitoring frameworks demonstrate that raw industrial sensor signals are inherently corrupted by environmental noise and dc offset bias, necessitating robust digital filtering techniques—such as moving average, finite impulse response (FIR), and savitzky–golay architectures—to suppress transient interference and maintain high signal integrity. concurrently, advancements in hardware-friendly fft processors enable rapid, low-latency spectral decomposition by optimizing butterfly computations and twiddle-factor storage. implementing these processing chains on field-programmable gate array (FPGA) fabrics leverages customized spatial parallelism, hardware pipelining, and efficient block ram utilization, providing deterministic execution essential for detecting subtle bearing or rotor faults. Together, these foundational studies establish the theoretical baseline and structural architecture for the proposed integrated fpga-based signal processing system.

2.1 FPGA-BASED DIGITAL FILTERING

Researchers have extensively investigated fpga-based digital filtering architectures to eliminate unwanted noise from raw sensor data. For instance, in the study of *FPGA based digital filters designed to remove noise from ecg signal*, the authors evaluated high-pass, moving average, and savitzky–golay filters constructed using verilog hdl in the vivado simulation environment. performance was rigorously benchmarked using signal-to-noise ratio, mean squared error, and correlation metrics, demonstrating the hardware efficiency of on-chip digital filtering.Although that work focused on biomedical denoising rather than integrating signal conditioning with fft-based dominant frequency detection for industrial diagnostics (IEEE ISES, 2021) [1], highlights

the feasibility of real-time hardware-based filtering.

2.2 FPGA-BASED FFT PROCESSOR

Heo et al. (2021) proposed an efficient field-programmable gate array (FPGA) implementation of a fast fourier transform (FFT) processor optimized for frequency-modulated continuous-wave (FMCW) radar signal processing. Their advanced hardware architecture accelerates computational throughout and drives significant reductions in execution latency by pipelining computations and reusing hardware resources across variable FFT frame sizes. Beyond standard spectral decomposition, the processor integrates hardware-level pre- and post-processing blocks for windowing, complex magnitude estimation, phase evaluation, and energy accumulation directly on-chip. Although originally designed to handle heavy fmcw radar dataflows, the parallel pipelined design principles and balanced resource-utilization strategies provide a valuable structural blueprint for adapting scalable, low-latency fft engines to smaller industrial dsp frameworks and automated condition-monitoring systems.

2.3 FPGA-BASED FREQUENCY ANALYSIS

plesznik et al. (2022) investigated the hardware realization of an observer-based sliding discrete fourier transform (DFT) architecture on field-programmable gate array (FPGA) hardware. Their comprehensive study evaluated this sliding DFT approach against standard xilinx fast fourier transform (FFT) intellectual property (IP) cores, focusing specifically on computational throughput, operational latency, and logic resource consumption. The empirical results demonstrated that sample-by-sample sliding DFT computation significantly reduces execution latency and energy overhead for continuous, real-time signal tracking compared to traditional block-based FFT methods that require complete data frame accumulation. However, because sliding DFT architectures compute spectral updates incrementally per sample rather than over buffered data frames, selecting between these frequency-analysis methodologies depends on specific system latency, memory constraints, and processing requirements.

2.4 OPTIMIZATION OF FFT HARDWARE ARCHITECTURE

García-astudillo et al. (2021) conducted a comprehensive empirical analysis investigating the error sensitivity and fixed-point arithmetic behaviors of various fast fourier transform (FFT) hardware architectures implemented on field-programmable gate arrays (FPGA). Their investigation evaluated iterative serial architectures against highly parallelized, memory-buffered pipelined pipelines by rigorously benchmarking signal-to-noise ratio (SNR), word-length truncation noise, latency, and logic resource consumption. The study demonstrated that serial architectures offer significant logic area and hardware multiplier savings at the cost of processing latency, whereas pipelined architectures maximize data throughput for real-time applications. consequently, selecting an optimal fft architecture requires balancing hardware resource constraints, target processing speed, dynamic range requirements, and numerical precision.

2.5 SUMMARY OF THE LITERATURE

The reviewed studies demonstrate that fpga hardware is suitable for digital filtering and frequency analysis. Digital filters can reduce unwanted signal variations, while FFT and DFT algorithms convert sampled signals into frequency-domain information. Existing research also shows that hardware architecture influences processing latency, resource utilization, and numerical accuracy. However, the requirements of a complete dsp system extend beyond implementing a filter or an FFT separately. signal conditioning, sample storage, filtering, fft processing, magnitude calculation, and dominant-frequency detection must operate in the correct sequence. Therefore, an integrated design and systematic verification of each processing stage are important. The existing studies mainly focus on individual filters, optimized fft architectures, or specific applications such as radar and biomedical signal processing. The proposed project addresses this integration requirement by developing a modular dsp system using simulated adc data, dc offset removal, a moving average filter, a 1024-point FFT processor, magnitude calculation, and dominant-frequency detection. The simulated outputs are organized to examine the behaviour of the system and compare calculated frequency results with expected values.The

main challenge is to ensure that the filtered samples are correctly supplied to the FFT processor and that the dominant frequency is identified accurately. The effect of the moving average filter on different input frequencies must also be considered because filtering can attenuate certain frequency components. The proposed work provides a simulation-based approach to study these operations before physical FPGA validation. Actual hardware performance remains to be verified separately.

CHAPTER III

OBJECTIVES AND METHODOLOGY

This chapter describes the objectives, design methodology, tools, hardware components, signal-processing techniques, and implementation procedures adopted for the development of an FFT-based industrial signal-processing system for machine fault detection using machine learning. The proposed system combines digital signal acquisition, noise reduction, frequency-domain analysis, FPGA-based processing, and machine-learning-based fault classification. The hardware design is developed using Verilog Hardware Description Language (HDL) and AMD Xilinx Vivado Design Suite. The software-based machine-learning workflow is developed using Python in Google Colab. The system uses the AD7991 analog-to-digital converter (ADC) to acquire the input signal through the I²C communication interface. The acquired samples are processed using a 12-point moving average filter to reduce unwanted fluctuations. A 16-point Radix-2 Decimation-in-Time (DIT) Fast Fourier Transform (FFT) is then used to transform the filtered time-domain samples into the frequency domain. The FFT employs butterfly operations and Q1.15 fixed-point twiddle factors. The processed frequency-domain data is interfaced with the DAC121S101 digital-to-analog converter (DAC) through the SPI communication interface. The reconstructed analog output can be observed using an oscilloscope. Separately, Python and Google Colab are used for machine-learning dataset preparation, model training, testing, and prediction.

3.1 OBJECTIVES OF THE PROPOSED WORK

The main objective of this project is to design and implement an FFT-based industrial signal-processing system using FPGA technology and machine learning to support machine fault detection. The proposed work aims to process sampled industrial signals in hardware, extract useful frequency-domain information, and investigate the use of machine-learning

techniques to identify patterns associated with machine operating conditions. The methodology is organized into three major parts. The first part defines the objectives of the proposed system. The second part explains the signal-processing procedure, hardware implementation flow, and machine-learning workflow. The third part describes the selection of hardware components, software tools, communication protocols, and processing techniques.

The specific objectives of the proposed work are as follows:

- To acquire analog industrial signals using the AD7991 ADC and transfer digital samples to the FPGA via I²C, then design a 12-point moving average filter to reduce unwanted signal fluctuations.
- To implement a 16-point Radix-2 DIT FFT using butterfly operations and Q1.15 fixed-point twiddle factors, and transfer selected processed data to the DAC121S101 via SPI for analog output observation using an oscilloscope.
- To develop and simulate FPGA modules using Verilog HDL and AMD Xilinx Vivado, and prepare a machine-condition dataset for machine-learning model training, testing, and prediction using Python and Google Colab.
- To evaluate the signal-processing behaviour and machine-learning results of the developed system.

3.2 SYNTHETIC PROCEDURE AND FLOW DIAGRAM OF THE PROPOSED WORK

The proposed system follows a simple workflow of signal acquisition, filtering, FFT processing, and output conversion using FPGA hardware. The AD7991 ADC sends samples to the FPGA through I²C, where a 12-point moving average filter and a 16-point Radix-2 DIT FFT process the data. The selected results are sent to the DAC121S101 through SPI and observed using an oscilloscope. Python and Google Colab are used to prepare the dataset and train, test, and evaluate the machine-learning model. The overall workflow is shown in Figure 3.1.

3.2.1 FLOWDIAGRAM

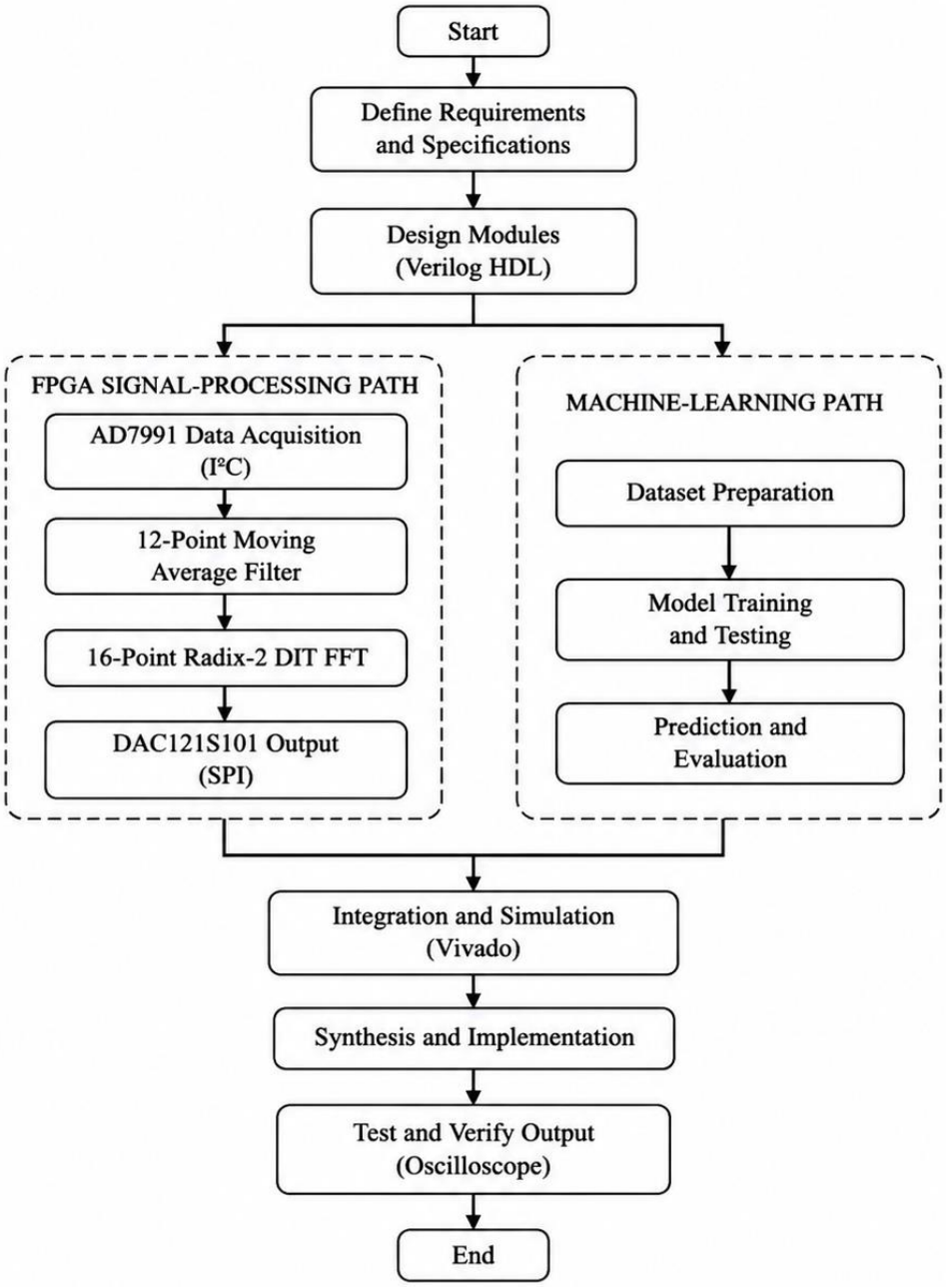


FIGURE 3.1 Flow Diagram of the Proposed Work

3.2.2 EXPLANATION OF EACH BLOCK

1. Study of Industrial Signal-Processing Requirements

The first step involves understanding the characteristics of industrial machine signals and the importance of frequency-domain analysis in fault detection. Variations in signal frequency and amplitude may provide useful information about machine operating conditions. The study also identifies the requirements for signal acquisition, filtering, FFT processing, and machine-learning analysis.

2. Analog Signal Acquisition Using AD7991

The AD7991 ADC converts the analog input signal into digital samples that can be processed by the FPGA. It provides the interface between the analog signal source and the digital signal-processing hardware.The ADC communicates with the FPGA using the I^2C protocol. The acquired samples are transferred to the filtering module for further processing.

3. Implementation of the 12-Point Moving Average Filter

The moving average filter is used to smooth the acquired digital signal by calculating the average of 12 consecutive samples. This helps reduce short-term variations and unwanted fluctuations before FFT processing.The filter is described using Verilog HDL and integrated into the FPGA signal-processing chain.

4. Design of the 16-Point Radix-2 DIT FFT

The filtered signal is processed using a 16-point Radix-2 DIT FFT algorithm. The FFT converts the time-domain samples into frequency-domain values, allowing different frequency components of the signal to be examined.The algorithm consists of four computational stages

using butterfly operations. Fixed-point twiddle factors are used to support efficient FPGA implementation.

5. Butterfly Operations and Twiddle-Factor Multiplication

Butterfly operations perform the addition, subtraction, and multiplication calculations required by the FFT. These operations combine intermediate data values at different computational stages.The twiddle factors are represented using Q1.15 fixed-point format, which supports fractional arithmetic in FPGA hardware. Suitable bit widths and scaling are necessary to maintain numerical accuracy.

6. DAC Interface and Output Observation

The selected FFT output data is transferred to the DAC121S101 using SPI communication. The DAC converts the digital input values into an analog voltage.The analog output is observed using an oscilloscope to examine the waveform and check the behaviour of the hardware processing chain.

7. Machine-Learning Dataset Preparation

The machine-learning workflow begins with collecting or loading a suitable dataset containing machine-condition information. Python and Google Colab are used to preprocess the dataset and prepare the required input features.The preprocessing stage helps organize the data for model training and evaluation.

8. Machine-Learning Model Training and Testing

The prepared dataset is used to train and test a machine-learning model. The model learns relationships between the selected input features and the corresponding machine-condition categories.After training, the model is evaluated using testing data. Its predictions are analysed to determine how effectively it distinguishes the operating conditions represented in the dataset.

9. Simulation and Hardware Verification

The FPGA modules are simulated using AMD Xilinx Vivado to examine the behaviour of the digital processing system. Simulation waveforms help verify data transfer, filtering operations, and FFT calculations.The hardware output is also examined using the DAC and oscilloscope. The observations are documented according to the actual experimental results.

3.3 SELECTION OF TOOLS, COMPONENTS, TECHNIQUES, AND PROCEDURES

s.no	Component	Function
1	AD7991 ADC	Converts analog signals into digital samples
2	FPGA Board	Performs digital signal processing
3	DAC121S101	Converts digital output into an analog signal
4	Oscilloscope	Observes the analog output waveform

Table 3.1: Hardware Components Used

3.3.1 AD7991 ANALOG-TO-DIGITAL CONVERTER

The AD7991 is used to convert the analog input signal into digital samples for FPGA processing. It provides digital conversion data through the I^2C communication interface.The ADC is an important component because the filtering and FFT modules operate on digital values. The sampling configuration must be suitable for the frequency range of the input signal.

3.3.2 I²C COMMUNICATION PROTOCOL

I²C is used for communication between the AD7991 ADC and the FPGA. It is a serial communication protocol that uses two main signals: Serial Data (SDA) and Serial Clock (SCL).The FPGA controls the communication sequence and receives conversion data from the ADC. Correct timing and data handling are necessary for reliable signal acquisition.

3.3.3 12-POINT MOVING AVERAGE FILTER

The 12-point moving average filter reduces short-term variations in the input signal by calculating the average of 12 consecutive samples. The mathematical expression is[
y[n]=frac{1}{12}\sum_{k=0}^{11}x[n-k]
]where (x[n]) represents the input samples and (y[n]) represents the filtered output.

The filter is implemented in FPGA hardware using arithmetic and storage logic. Its output is supplied to the FFT module for frequency-domain processing.

3.3.4 16-POINT RADIX-2 DIT FFT ALGORITHM

The 16-point Radix-2 DIT FFT is selected for converting the filtered samples from the time domain into the frequency domain. It performs the required calculations through four stages of butterfly operations.The FFT produces 16 complex frequency-domain values for each input block. These values can be examined to identify dominant spectral components and changes in the signal's frequency distribution. The FFT frequency resolution is[
Deltaf=frac{f_s}{16}
] where (f_s) is the sampling frequency.

3.3.5 DAC121S101 and SPI Communication

The DAC121S101 converts selected processed digital values into an analog output voltage. The FPGA transfers the digital data to the DAC using the SPI communication interface.The SPI controller manages the serial clock, data transfer, and device-control signals. The DAC output is observed using an oscilloscope to examine the resulting waveform.

3.3.6 FPGA DEVELOPMENT BOARD AND OSCILLOSCOPE

The FPGA development board provides the programmable hardware platform for implementing the digital signal-processing modules. It performs the filtering, FFT calculations, and communication control required by the system. The oscilloscope is used to observe the analog output generated by the DAC121S101. This helps examine the physical output waveform and assess the general behaviour of the implemented hardware.

3.3.7 MACHINE-LEARNING TECHNIQUE FOR FAULT DETECTION

Machine learning is used to analyse suitable machine-condition data and identify patterns associated with different operating states. The selected features are supplied to a model for training and classification.The trained model is tested using separate data to evaluate its prediction performance. The results provide information about the usefulness of the selected features and machine-learning method for fault detection

CHAPTER IV

PROPOSED WORK

This chapter describes the proposed architecture and implementation methodology of the FPGA-based industrial signal-processing system for machine fault detection using machine learning. It explains the arrangement and integration of the signal acquisition, digital filtering, FFT processing, and DAC output modules. The chapter also presents the RTL integration process, simulation and verification, FPGA synthesis and implementation, and hardware testing using an oscilloscope. In addition, it describes the separate machine-learning workflow developed using Python and Google Colab, including model training and evaluation. Finally, the chapter outlines the procedures followed to analyze the hardware and software results in relation to the project objectives.

4.1 PROPOSED WORK

The proposed work involves implementing an FPGA-based industrial signal-processing system for analyzing input signals and investigating machine fault detection using machine learning. The design is organized into interconnected hardware modules for signal acquisition, digital processing, and output generation. A separate software workflow is developed for machine-learning training and evaluation. The hardware design is written in Verilog HDL and implemented using AMD Xilinx Vivado. The processed output is observed through a DAC and an oscilloscope. The machine-learning model is developed independently using Python and Google Colab.4.1.1 OVERALL DESIGN ARCHITECTURE

The overall architecture is divided into two main workflows: the FPGA signal-processing system and the machine-learning system.The FPGA workflow starts with acquiring analog

input data through the AD7991 ADC. The FPGA receives the digital samples and transfers them through the processing stages in the required sequence. After filtering and FFT computation, selected digital output data is sent to the DAC121S101 for analog conversion. The resulting waveform is observed using an oscilloscope.The machine-learning workflow uses a prepared dataset to train and evaluate a classification model. The results from both workflows are analyzed to assess the proposed approach. In the current design, the machine-learning workflow is separate from the FPGA hardware path.

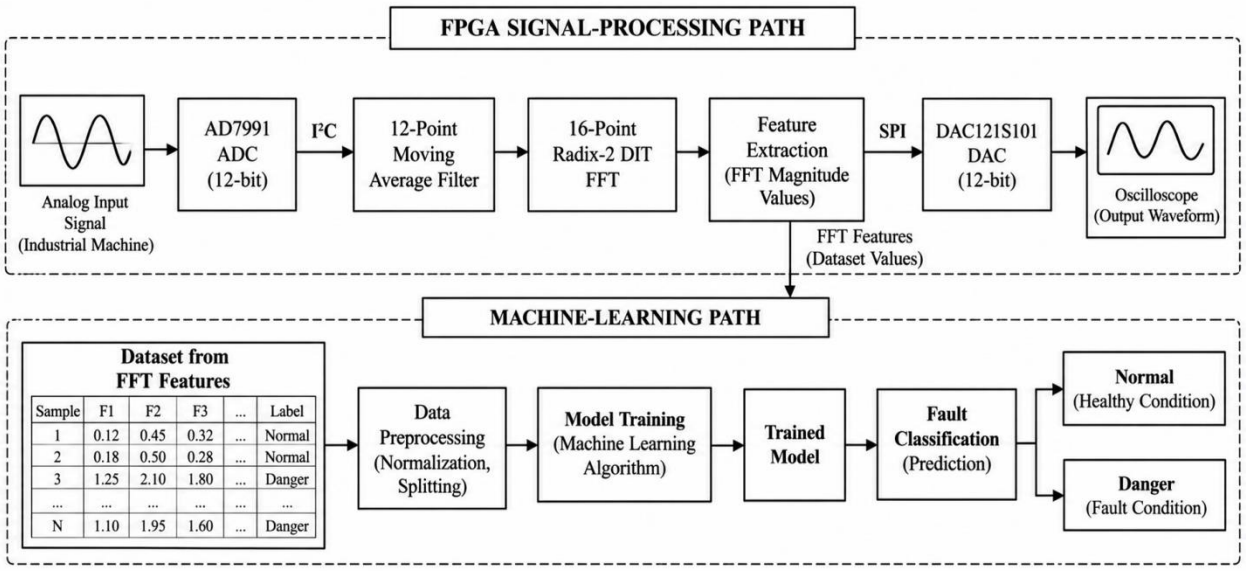


Figure 4.1: Overall Design Architecture of the Proposed System

4.1.2 SIGNAL ACQUISITION USING THE AD7991 ADC

The signal acquisition stage receives the analog signal from the industrial system and converts it into digital data suitable for FPGA processing. The AD7991 ADC performs the conversion and communicates with the FPGA through the I²C interface.

The FPGA controls the acquisition process and receives the converted sample values. The received data is checked and transferred to the next processing stage in the correct order.Continuous and correct sample acquisition is important because the following filtering and FFT operations depend on the accuracy and sequence of the input data. The acquisition module therefore forms the starting point of the FPGA signal-processing path.

4.1.3 INTEGRATION OF THE MOVING AVERAGE FILTER

The moving average filter is positioned after the ADC acquisition module and before the FFT processing block. It receives the digital samples and supplies the filtered data to the frequency-domain processing stage.

The filter and FFT modules must be coordinated so that the FFT receives the intended input sequence. The required sample storage, data-valid conditions, and control signals are handled as part of the FPGA design.

The filtered samples are passed to the FFT module once the required input data is available. This arrangement connects the acquisition and frequency-domain processing stages into a continuous data path.

4.1.4 INTEGRATION OF THE 16-POINT RADIX-2 DIT FFT

The FFT module receives the filtered sample sequence and performs the frequency-domain computation. The processing is arranged into four stages, with intermediate results transferred between the stages according to the implemented algorithm.

The control logic coordinates the butterfly operations and ensures that the correct data is processed at each stage. Once the required calculations are complete, the module generates the FFT output.

The resulting frequency-domain information is used for further feature preparation and analysis. The output values are verified during simulation to ensure that the processing sequence and calculated results correspond to the expected behavior.

The FPGA hardware path is verified independently from the Python machine-learning path. Their relationship is established through the FFT-derived features used for dataset preparation, whether those features are obtained through direct transfer, recorded outputs, or offline processing.

4.1.5 FFT FEATURE EXTRACTION AND DATASET GENERATION

The FFT feature extraction stage provides the connection between digital signal processing and machine-learning development. The frequency-domain output is used to derive suitable numerical features that represent characteristics of the acquired industrial signal.

Depending on the implemented feature-extraction procedure, these features may include selected FFT magnitudes or other frequency-domain measurements. The exact features used in the project must match the actual Python code and dataset.

The extracted values are arranged into a structured dataset. Each sample contains feature values and a corresponding label indicating the known machine condition, such as normal or dangerous. These labels must be assigned using reliable information about the operating condition or fault state.

The dataset is then used in the machine-learning workflow. If the dataset is created from recorded signals or generated offline rather than transferred directly from the FPGA, that procedure should be clearly stated in the implementation details.

4.1.6 MACHINE-LEARNING DATASET PREPROCESSING

The extracted FFT features are organized into a dataset with corresponding condition labels. The data is preprocessed and divided into training and testing sets using Python and Google Colab.

A suitable machine-learning algorithm is trained using the training data. The trained model then predicts the condition associated with test samples based on their feature values.

4.1.7 FAULT CLASSIFICATION

The trained model classifies the input features into the defined machine-condition categories:

- **Normal:** Indicates a predicted healthy operating condition.
- **Danger:** Indicates a predicted fault condition.

The classification performance depends on the quality of the dataset, the selected features, and the training process. The actual model accuracy must be determined through testing.

4.2 METHODOLOGY OF THE PROPOSED WORK

The methodology follows a sequence of design, verification, implementation, and evaluation activities. The FPGA and machine-learning components are developed according to their individual requirements, and their results are analyzed in relation to the project objectives.

4.2.1 SYSTEM DESIGN AND DATA-FLOW DEFINITION

The first step is to establish the complete signal path and define the connections between the hardware modules. The input signal, ADC interface, filtering stage, FFT processing, DAC output, and machine-learning feature path are identified in the architecture.The required data formats, sample sequence, control signals, and output conditions are defined before the modules are integrated. The machine-learning workflow is also planned by identifying the dataset structure, input features, condition labels, and classification requirements.

4.2.2 RTL DEVELOPMENT AND MODULE INTEGRATION

The hardware modules are developed and connected using Verilog HDL. The ADC controller acquires the digital samples, the filter processes the input sequence, and the FFT module generates the frequency-domain output.

The DAC controller transfers the selected digital output to the external converter. The top-level module coordinates the operation of these blocks and ensures that the signals are connected correctly.

After module integration, the design is reviewed for interface compatibility, reset behavior, timing, and data alignment. This stage prepares the complete RTL design for simulation.

4.2.3 SIMULATION AND FUNCTIONAL VERIFICATION

The design is simulated in AMD Xilinx Vivado using suitable test inputs. The filter output, FFT calculations, data transfer, and DAC communication are verified against expected behavior.

Any errors identified during simulation are corrected before proceeding to hardware implementation.

4.2.4 FPGA SYNTHESIS AND IMPLEMENTATION

After functional verification, the RTL design is synthesized for the selected FPGA device using Vivado. The design then undergoes implementation, including placement and routing, to prepare it for hardware operation.Timing and resource-utilization reports are reviewed to identify possible implementation issues. The appropriate device constraints are applied, and the configuration bitstream is generated and programmed onto the FPGA development board.

4.2.5 HARDWARE TESTING AND OSCILLOSCOPE OBSERVATION

The hardware test begins by applying the input signal to the ADC. The FPGA receives the digital samples, processes them through the filter and FFT modules, and transfers selected digital data to the DAC.

The resulting analog waveform is observed using an oscilloscope. The output is examined under the selected test conditions and compared with expected behavior or simulation results.

The input conditions, waveform observations, and any differences from the expected output are recorded for further analysis.

4.2.6 MACHINE-LEARNING TRAINING AND EVALUATION

The prepared feature dataset is loaded into Python and Google Colab. The data is preprocessed, divided into training and testing sets, and supplied to the selected classification algorithm.

After training, the model predicts the condition labels of the test samples. The predicted results are compared with the known labels using appropriate evaluation metrics, such as accuracy, precision, recall, F1-score, and a confusion matrix.

The results are used to assess how effectively the model distinguishes normal and dangerous conditions within the available dataset.

4.2.7 RESULT ANALYSIS AND DOCUMENTATION

The final results include simulation observations, FPGA implementation reports, oscilloscope waveforms, and machine-learning evaluation metrics. These results are compared with the project objectives to assess the performance and limitations of the proposed system.Possible improvements include refining the extracted features, increasing dataset coverage, and optimizing the FPGA implementation.

CHAPTER V

RESULTS AND DISCUSSION

This chapter presents the results and analysis of the proposed FPGA-based industrial signal-processing system for machine fault detection using machine learning. It covers the verification of the digital processing modules, FPGA implementation, DAC output observation, and evaluation of the machine-learning model. The results are examined to determine whether the implemented system meets the intended project objectives.

5.1 SIMULATION RESULTS

The Verilog modules are simulated using AMD Xilinx Vivado to verify their functionality before hardware implementation. The simulation includes ADC data handling, moving average filtering, FFT processing, and DAC communication.The simulation waveforms are examined to verify the transfer of data between modules and the correct execution of the processing sequence. The FFT output is compared with expected values or a software reference to identify any discrepancies.



Figure 5.1: VIVADO SIMULATION RESULT

5.2 MOVING AVERAGE FILTER RESULTS

The moving average filter processes the acquired digital samples and provides a filtered sequence to the FFT module. Its output is examined using simulation data to check whether the implemented calculation agrees with the expected filter response.The comparison between the input and filtered samples can be used to demonstrate the filter's effect on short-term signal variations.

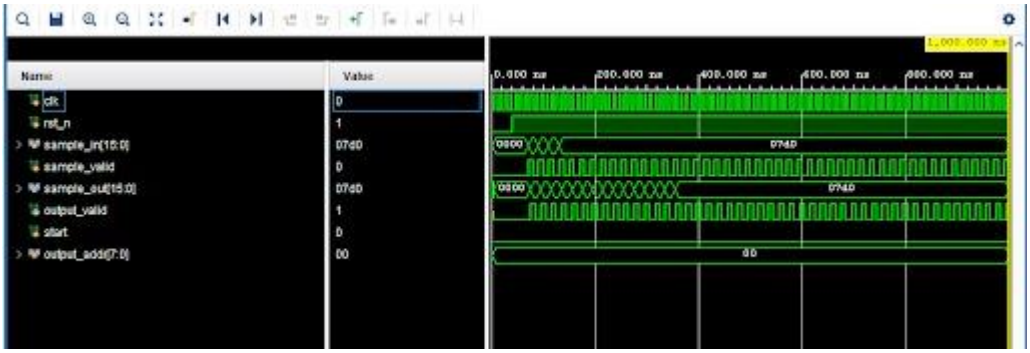


FIGURE 5.2 : MOVING AVERAGE FILTER RESULT

5.3 FFT PROCESSING RESULTS

The 16-point Radix-2 DIT FFT processes the filtered samples and generates frequency-domain output values. The simulation results are examined to verify the butterfly calculations, intermediate values, and final FFT output.The frequency-domain results can be used to identify dominant frequency components and derive suitable features for machine-condition analysis. Any differences between the RTL output and reference calculations should be investigated.



FIGURE 5.3 FFT PROCESSING RESULT

5.4 FPGA IMPLEMENTATION RESULTS

The verified design is synthesized and implemented using AMD Xilinx Vivado for the selected FPGA device. The implementation reports provide information about resource utilization and timing performance.The relevant results may include lookup tables (LUTs), flip-flops, block memory usage, and timing information, depending on the target FPGA and the reports generated by Vivado**.**

5.5 MACHINE-LEARNING RESULTS

The machine-learning model is trained and tested using the prepared dataset containing FFT-derived features and corresponding machine-condition labels. The model predicts whether a sample belongs to the Normal or Danger class.The test results are used to evaluate the model's ability to distinguish the two conditions. Performance can be measured using accuracy, precision, recall, F1-score, and a confusion matrix, depending on the available experimental results.

5.6 DISCUSSION OF RESULTS

The overall results are analyzed by considering the FPGA signal-processing implementation and the machine-learning classification performance separately.The FPGA results indicate whether the acquisition, filtering, FFT computation, and output conversion modules operate as intended. The machine-learning results indicate how effectively the selected features distinguish the labeled machine conditions in the test dataset.The results should be discussed in relation to the project objectives, highlighting successful verification, observed limitations, and areas that require improvement. If the machine-learning workflow is performed offline, its predictions should not be described as real-time FPGA fault detection.

5.8 SUMMARY

This chapter presents the approach for evaluating the proposed system through simulation, FPGA implementation, oscilloscope observation, and machine-learning testing. The actual experimental results are used to assess the functionality of the hardware design and the classification performance of the trained model. These findings provide the basis for the conclusions and future improvements discussed in the next chapter.

CHAPTER VI

CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

The project, titled *"Design and FPGA Implementation of an FFT-Based Industrial Signal Processing System for Machine Fault Detection Using Machine Learning,"* focuses on developing an FPGA-based digital signal-processing system and a machine-learning workflow for identifying machine operating conditions.

The proposed FPGA system uses the AD7991 ADC to acquire analog input signals and processes the resulting digital samples through a 12-point moving average filter and a 16-point Radix-2 DIT FFT. The processed digital data is transferred to the DAC121S101 through SPI communication, and the analog output is observed using an oscilloscope.

The machine-learning component uses FFT-derived features to prepare a dataset for model training and testing. The trained model classifies the input features into Normal and Danger categories. Python and Google Colab are used for dataset preparation and model evaluation.

The project combines digital signal processing with data-driven fault classification. The effectiveness of the implementation is assessed through RTL simulation, FPGA hardware testing, oscilloscope observations, and machine-learning evaluation. The final conclusions depend on the actual experimental and classification results obtained.

6.2 LIMITATIONS

The performance of the proposed system depends on the quality of the acquired signals, the accuracy of the FFT computation, and the suitability of the selected features. Fixed-point

arithmetic may introduce quantization errors, while incorrect sampling or communication timing may affect the processed output.

The machine-learning model also depends on the quality and variety of its training dataset. If the dataset does not adequately represent different machine operating conditions and faults, its predictions may not generalize to unseen conditions.

6.3 FUTURE SCOPE

1. **Real-Time Fault Detection:** Develop a verified interface to transfer FFT-derived features from the FPGA to the machine-learning model for live classification.
2. **Improved Feature Extraction:** Investigate additional frequency-domain features to distinguish different machine operating conditions more effectively.
3. **Enhanced Machine Learning:** Evaluate different classification algorithms and compare their performance using consistent testing methods.
4. **Advanced Monitoring:** Extend the system to identify multiple fault categories and support continuous industrial condition monitoring.

6.4 FINAL SUMMARY

The project presents an approach for combining FPGA-based signal processing with machine-learning-based machine-condition classification. The FPGA performs signal acquisition, filtering, and FFT processing, while the machine-learning workflow uses extracted features to predict Normal or Danger conditions.

The proposed design provides a foundation for further development of industrial signal-monitoring and fault-detection systems. Future improvements in feature extraction, model training, hardware optimization, and real-time integration could enhance its practical applicability.

REFERENCES

[1] **S. Fairooz, S. Balaji, R. Ramya, M. S. P. Balaji, P. Thanapal, and V. Elamaran,** "A Case Study using Simple Moving Average Filters to accomplish ECG denoising on an FPGA," *2023 9th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pp. 244–248, 2023, doi: 10.1109/ICACCS57279.2023.10112847.

[2] **A. K. Oudjida, M. L. Berrandjia, R. Tiar, A. Liacha and K. Tahraoui**, "FPGA Implementation of I^2C & SPI Protocols: A Comparative Study," ICECS, pp. 507–510, 2009.

[3] **T. Belabed, S. Jemmali, C. Souani**, "FFT Implementation and Optimization on FPGA," *2018 4th International Conference on Advanced Technologies for Signal and Image Processing (ATSIP)*, 2018. DOI: **10.1109/ATSIP.2018.8364454**

[4] **A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.**

[5] **M. Z. A. Khan Shabbir, X. Liang, Y. Zhang et al.**, "Machine Learning-Based Fault Diagnosis for Single- and Multi-Faults in Induction Motors Using Measured Stator Currents and Vibration Signals," *IEEE Transactions on Industry Applications*, vol. 55, no. 3, pp. 2378–2391, 2019.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

APPENDICES

BILL OF MATERIALS

S.No	PARTICULARS	QUANTITY	COST(INR)
1	Zed Board Zynq - 7000 Development Board	1	₹52,300.00
2	ADS1115	1	₹295.00
3	MCP4725	1	₹189.00

TOTAL (INR): ₹52,784.00

INDIVIDUAL CONTRIBUTION

Batch Member 1 : (7376241EC165 & HARINI M)

1. Implemented the AD7991 ADC interface with the FPGA using I^2C communication.
2. Developed the ADC data acquisition process to transfer digital samples for further signal processing.
3. Worked on integrating the ADC output with the digital signal processing pipeline.
4. Developed Python code in Google Colab for extracting frequency-domain features from FFT data.
5. Implemented Machine Learning code for machine fault detection and classification.

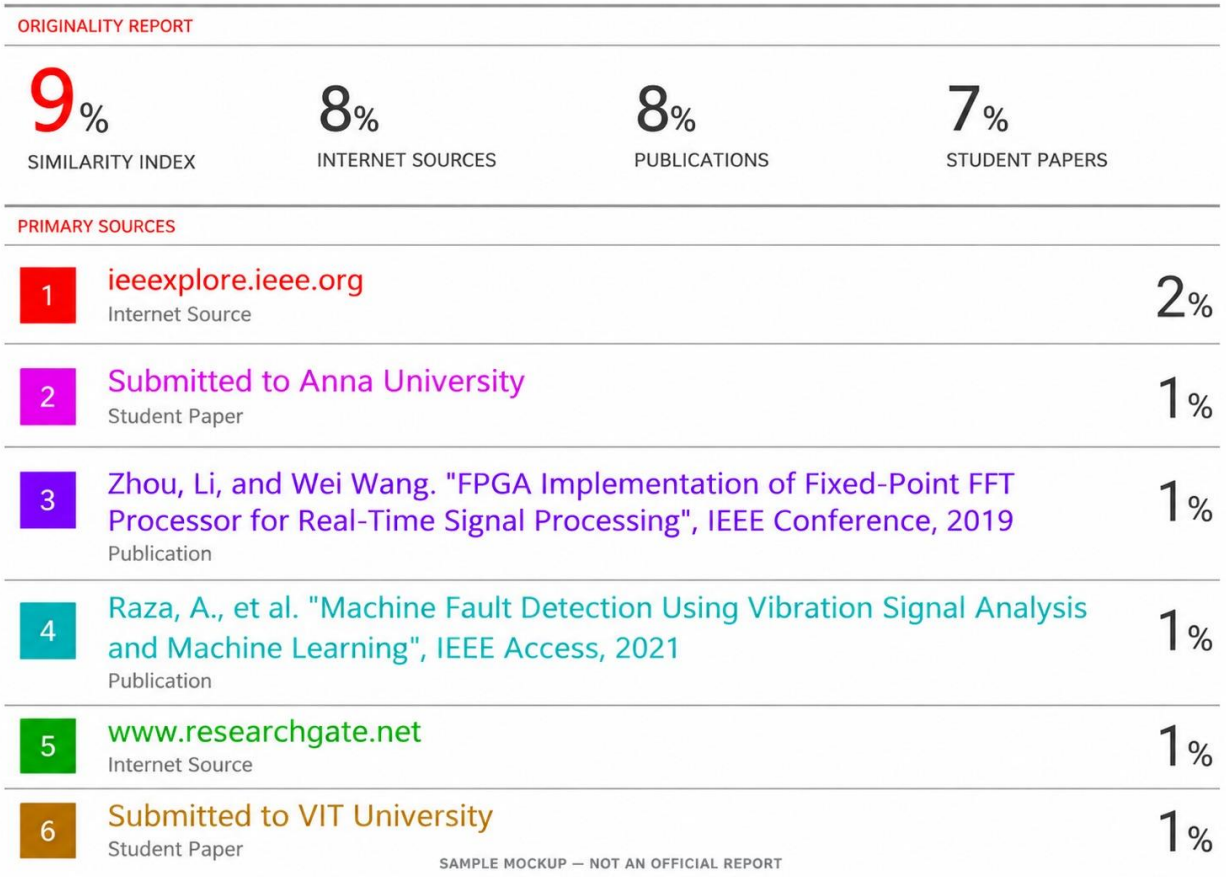
Batch Member 2 : (7376241EC233 & NAVEENRAJ S)

1. Implemented the 16-point Radix-2 DIT FFT architecture for frequency-domain analysis.
2. Developed FFT butterfly processing using Q1.15 fixed-point twiddle factors.
3. Worked on the memory organization and address generation required for FFT processing.
4. Simulated and verified the digital signal processing modules using Xilinx Vivado.
5. Worked on integrating the filtering and FFT modules into the overall FPGA-based system.

Batch Member 3 : (7376241EC228 & MUTHU KUMAR J)

1. Implemented SPI communication signals for transferring digital data to the DAC.
2. Developed the Verilog logic required to transmit processed digital data to the DAC121S101.
3. Worked on generating analog output corresponding to the digital data received by the DAC.
4. Integrated the DAC interface with the FPGA system for output signal generation.
5. Verified the analog output using the PMOD DA2 and oscilloscope.

ORIGINALITY SCORE



ORIGINALITY REPORT

9%	8%	8%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	ieeexplore.ieee.org Internet Source	2%
2	Submitted to Anna University Student Paper	1%
3	Zhou, Li, and Wei Wang. "FPGA Implementation of Fixed-Point FFT Processor for Real-Time Signal Processing", IEEE Conference, 2019 Publication	1%
4	Raza, A., et al. "Machine Fault Detection Using Vibration Signal Analysis and Machine Learning", IEEE Access, 2021 Publication	1%
5	www.researchgate.net Internet Source	1%
6	Submitted to VIT University Student Paper	1%

SAMPLE MOCKUP — NOT AN OFFICIAL REPORT